

MEMORY SYSTEM DESIGN FOR AI & ML ENGINEERS

Production Memory Engineering for Agentic AI

An Engineering Guide to State Management, Episodic Long-Term Memory, Multi-Tenant Context Optimization, and Interview Preparation

Lamhot Siagian • AI Engineering Insider

Write & read paths, salience and forgetting curves, pgvector retrieval, consolidation, multi-tenant isolation, evaluation, and 130 interview questions --- grounded in one runnable reference stack.



Production Memory Engineering for Agentic AI.

© 2026 Lamhot Siagian / AI Engineering Insider. All rights reserved.

This book accompanies the open `memory-rag` reference project (FastAPI + LangGraph + pgvector backend, Next.js lab UI). Every code listing is drawn from or directly runnable against that project, and every chapter can be verified interactively through its matching lab in the web UI.

Code samples are provided for study and reference. Model names, library versions, and external service behaviour change over time; verify against current documentation before production use.

Contact: aiengineeringinsider.substack.com

Contents

How to Use This Book	vii
----------------------	-----

Part I Foundations	1
--------------------	---

1 Foundations — Why Memory Layers Exist	2
---	---

1.1 Retrieval Is Not Memory	2
1.2 Where Memory Sits in the RAG Pipeline	3
1.3 The Stateless Failure, Demonstrated	4
1.4 Production Code: An Agent with Two Read Paths	4
1.5 Test It in the UI	6
1.6 Evaluation	7

2 Types of Memory	11
-------------------	----

2.1 Short-Term vs. Long-Term	11
2.2 The Four Long-Term Types	11
2.3 Architecture: the Memory Schema	13
2.4 Production Code: One Table, Four Types	13
2.5 Test It in the UI	14
2.6 Evaluation	15

3 Memory Storage Architectures	19
--------------------------------	----

3.1 Three Questions, Three Store Types	19
3.2 Hybrid and Tiered Storage	20
3.3 Graph-Based Memory: Knowledge Graph Triples	20
3.4 Architecture: the Reference Hybrid	23
3.5 Production Code: Embeddings, Chunking, Indexing	23
3.6 Test It in the UI	24
3.7 Evaluation	25

Part II The Memory Pipeline	29
-----------------------------	----

4 Memory Write Path	30
---------------------	----

4.1 When to Write: Every Turn vs. Checkpoints	30
4.2 Extraction: Conversation → Structured Units	30
4.3 Salience: What Is Worth Remembering	31
4.4 Dedup and Indexing: the Atomic Write	32
4.5 Forgetting: Decay Over Time	33
4.6 Architecture of the Write Path	34
4.7 Test It in the UI	34

4.8	Evaluation	35
5	Memory Read Path — Retrieval Strategies	38
5.1	Four Strategies, One Interface	38
5.2	The Scoring Math	39
5.3	Architecture of the Read Path	40
5.4	Production Code: the Retriever	40
5.5	Test It in the UI	42
5.6	Evaluation	44
Part III	Lifecycle & Multi-Tenancy	48
6	Memory Consolidation, Conflict Resolution & Memory Updates	49
6.1	Why Consolidate?	49
6.2	Hierarchical Summarisation	49
6.3	Architecture of the Consolidation Loop	50
6.4	Production Code: Compress and Prune	50
6.5	Conflict Detection & Resolution	51
6.6	Self-Improving Memory & Active Reflection Loops	52
6.7	Test It in the UI	54
6.8	Evaluation	55
7	Context Window & Injection Strategy	58
7.1	When to Inject: Always vs. Conditionally	58
7.2	How to Inject: Structured Context Blocks	58
7.3	The Token Budget Problem	59
7.4	Architecture of the Injection Pipeline	60
7.5	Test It in the UI	60
7.6	Evaluation	61
8	Personalization & Multi-User Design	65
8.1	Identity Is the Partition Key	65
8.2	Isolation by Construction	66
8.3	Private vs. Shared Memory	66
8.4	Multi-Agent Shared Memory & Blackboard Patterns	67
8.5	Architecture: Multi-Tenant Memory	69
8.6	Production Code: Scoped Retrieval	69
8.7	Test It in the UI	70
8.8	Evaluation	70
Part IV	Quality, Governance & Production	74
9	Evaluation	75
9.1	The Four Metrics	75
9.2	Architecture of an Evaluation Harness	76
9.3	Production Code: the Evaluation Endpoint	76
9.4	Long-Horizon Consistency	77
9.5	Test It in the UI	78

9.6	Synthetic Users and Memory Stress Testing	78
9.7	Evaluation	80
10	Privacy, Security & Governance	84
10.1	The Three Rights	84
10.2	Right to Be Forgotten: Cascade Across Stores	84
10.3	Data Portability: Export	85
10.4	Encryption & Access Control	86
10.5	Retention	87
10.6	Architecture of the Governance Layer	87
10.7	Test It in the UI	87
10.8	Evaluation	88
11	Reference Architectures & Frameworks	92
11.1	Generative Agents — the Memory Stream	93
11.2	MemGPT / Letta — Memory as an Operating System	93
11.3	Mem0 and Zep — Memory as a Service	93
11.4	Framework Memories — LangChain & LlamaIndex	94
11.5	LongMem — Decoupled Memory Networks	94
11.6	Model Context Protocol (MCP) for Memory Integration	94
11.7	Memory Reasoning Graphs	95
11.8	Architecture: Positioning the Reference Project	97
11.9	Test It in the UI	97
11.10	Evaluation	98
12	Scalability & Production Concerns	102
12.1	Sharding by Tenant	102
12.2	Asynchronous, Non-Blocking Writes	103
12.3	Caching the Hot Set	104
12.4	Controlling Embedding Cost	105
12.5	Long-Context Transformers vs. Memory RAG	105
12.6	Observability: Tracing Memory with LangSmith	106
12.7	Architecture: the Scaled Write Path	107
12.8	Test It in the UI	107
12.9	Evaluation	108
Part V	Capstone	112
13	Capstone — The Full Memory System	113
13.1	The Whole System in One View	113
13.2	A Fact’s Journey, End to End	113
13.3	Run It Yourself: the Multi-User Capstone	114
13.4	Extend It: Build-It-Yourself Projects	115
13.5	Evaluation	116
14	Case Studies: 20 Production Memory Architectures	120
14.1	20 Production Case Studies	120
14.2	Architecture: Tiered Memory Routing	121
14.3	Production Code: Tiered Memory Gateway	122

14.4	Test It in the UI	123
14.5	Evaluation	124
15	References	126

How to Use This Book

Subscribe → aiengineeringinsider.substack.com/subscribe

This is a *hands-on system-design* book. It is built around one reproducible project — the [memory-rag](#) application — and teaches production RAG memory by dissecting that project module by module. Every chapter follows the same rhythm:

1. **Overview** — the concept, the trade-offs, and where it sits in the memory pipeline.
2. **Architecture diagram** — a TikZ view of the components and data flow for that chapter.
3. **Production code** — the real backend module that implements the concept, explained line by line.
4. **Test it in the UI** — the exact lab route to open and the result you should observe, so the theory is verifiable, not abstract.
5. **Evaluation** — ten interview-grade questions (conceptual, debugging, and production) with full worked answers.

★ Key Insight

The fastest way to learn this material is to read a chapter, then immediately open the matching lab (Table 1) and reproduce the result. Memory systems are full of subtle feedback loops; seeing the numbers move in the UI builds intuition that prose cannot.

Running the reference project

Shell: bring up the memory-rag project

```
# 1. Backend (FastAPI + LangGraph + pgvector)
ollama pull llama3.1 && ollama pull nomic-embed-text
docker compose up -d          # Postgres + pgvector
cd backend && pip install -r requirements.txt
uvicorn app.main:app --reload  # http://localhost:8000/api/v1

# 2. Frontend lab UI (Next.js)
cd frontend && npm install && npm run dev    # http://localhost:3000
```

Chapter map

Table 1 lists every chapter, its core focus, the backend module it dissects, and the lab route that lets you test it live.

#	Chapter focus	Backend module dissected	UI lab
1	Foundations — why memory exists	chat/langgraph_agent.py, tools.py	/labs/1
2	Types of memory	db/models.py	/labs/2
3	Storage architectures	db/pgvector_utils.py	/labs/3
4	Memory write path	memory/writer.py	/labs/4
5	Memory read path (retrieval)	memory/reader.py	/labs/5
6	Consolidation, Conflict Resolution & Updates	memory/consolidator.py	/memory
7	Context window & injection	memory/context_manager.py	/labs/7
8	Personalization & multi-tenant	auth/, memory/service.py	/labs/8
9	Evaluation	memory/routes.py (evaluate)	/labs/9
10	Privacy, security & governance	memory/privacy.py	/labs/10
11	Reference architectures	— (survey)	/labs/11
12	Scalability & production	db/, async write paths	/labs/12
13	Hands-on capstone	full stack	/labs/13
B	Bonus: 20 production systems	— (case studies)	---

Table 1: Chapter map: each chapter dissects one module of the `memory-rag` project and is verified through a matching lab route in the UI.

About this edition

This build includes the full scaffold, all thirteen foundations-to-capstone chapters (1–13), and the Case Studies chapter (Chapter 14), each grounded in the real backend and verifiable through its matching lab in the UI.

PART I

Foundations

Before optimising a memory system you must understand what memory *is* in an agent, why stateless retrieval is not enough, and how the read and write paths fit together. Part I establishes that vocabulary and the mental model the rest of the book builds on.

Foundations — Why Memory Layers Exist

Subscribe → aiengineeringinsider.substack.com/subscribe

Retrieval tells an agent what is true in general. Memory tells it what is true about this user, right now. A production RAG system needs both.

A vanilla Retrieval-Augmented Generation (RAG) pipeline is stateless. It embeds a query, pulls the nearest chunks from a document index, stuffs them into the prompt, and answers. Ask it the same question in two different sessions and it behaves identically, because it carries nothing forward. That is fine for a documentation search box and fatal for an agent that is supposed to feel like it *knows* you.

This chapter defines the difference between **retrieval** and **memory**, shows precisely where memory plugs into the RAG pipeline, and grounds the distinction in the reference project’s LangGraph agent — which exposes *both* a document retriever and a personalized memory recall tool. By the end you will be able to articulate, at a system-design level, why memory is a separate subsystem and not just “more retrieval.”

1.1 Retrieval Is Not Memory

The two are easy to conflate because both end in “inject some text into the prompt.” The difference is in *what* the text represents and *how* it changes over time.

Retrieval vs. Memory

Retrieval reads from a (largely) static knowledge base — product docs, policies, a wiki. The same query returns the same chunks regardless of who is asking. It is shared, read-only, and corpus-scoped.

Memory reads and *writes* a dynamic, evolving, per-subject state — what the user told you, what the agent did, what preferences emerged. It is private, mutable, and identity-scoped.

Table 1.1 makes the contrast concrete along the dimensions that actually drive architecture decisions.

Dimension	Document retrieval	Agent memory
Source of truth	External corpus (docs)	The interaction history itself
Mutability	Read-only at query time	Read <i>and</i> write every turn
Scope	Shared across all users	Isolated per user / tenant
Lifetime	As long as the doc exists	Decays; can be consolidated or forgotten
Write trigger	Manual ingestion / indexing	Automatic extraction from dialogue
Ranking signal	Semantic similarity	Similarity <i>plus</i> recency & importance
Failure mode	Missing or stale documents	Stale facts, contradictions, leakage

Table 1.1: Retrieval and memory differ on every axis that matters for system design — which is why production agents implement them as separate subsystems.

★ Key Insight

If you remember one thing from this chapter: **retrieval is a read over a shared corpus; memory is a read–write loop over per-user state.** The moment you need writes, isolation, recency, or forgetting, you have left retrieval and entered memory — and you need the machinery in the rest of this book.

1.2 Where Memory Sits in the RAG Pipeline

Memory is not a single box bolted onto the end of RAG. It participates at three points: it can *reshape the query* before retrieval (pre-retrieval), it can *add a parallel context source* alongside document retrieval, and it *captures new state* after the answer is produced (post-response write). Figure 1.1 shows the full loop as implemented by the reference agent.

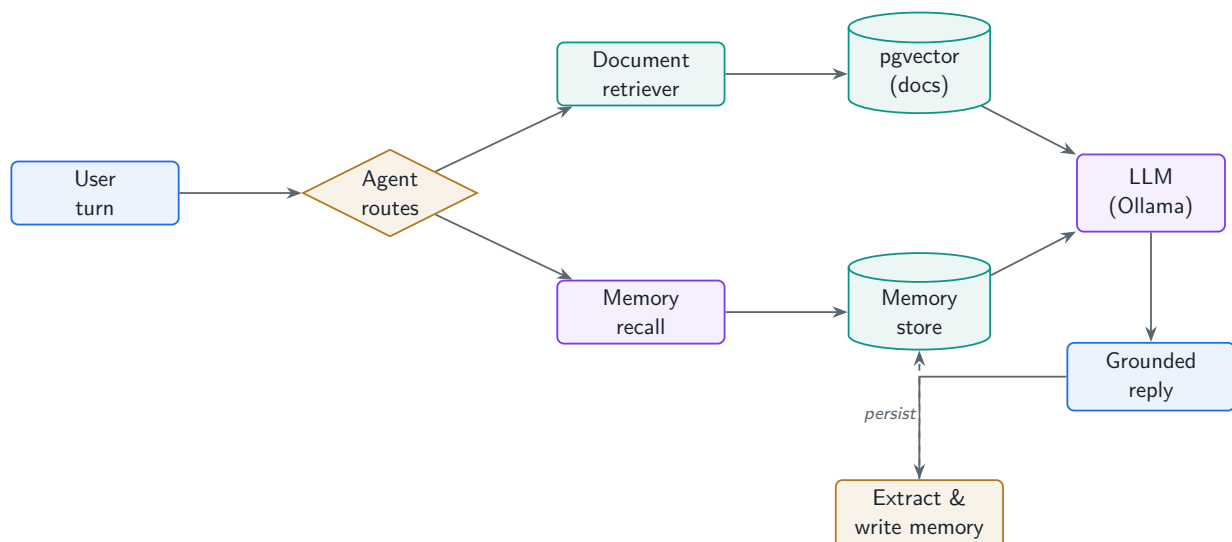


Figure 1.1: The memory-augmented RAG loop. Memory contributes a parallel read path (recall) before generation and a write path (extraction) after it, wrapping the classic document-retrieval pipeline.

The decision node is the heart of an agentic system: the model itself chooses *whether* to call the document retriever, the memory recall tool, both, or neither. That routing is what separates an *agent* with memory from a fixed RAG chain.

1.3 The Stateless Failure, Demonstrated

Consider the canonical failure. A user says, “Remember my production server runs on port 8080.” A turn later they ask, “Which port is my server on?” A stateless pipeline cannot answer: the first statement was never persisted and the second query retrieves nothing useful from a *document* index because the fact never lived in a document. It lived in the conversation.

! Pitfall / Warning

A longer context window does *not* fix this. Bigger windows enlarge *short-term* working memory within a single session, but the instant the session ends — or the window is trimmed — the fact is gone. Persistence is an architectural property, not a context-length setting.

1.4 Production Code: An Agent with Two Read Paths

The reference project builds a LangGraph ReAct agent whose tool list is exactly the two read paths from Figure 1.1: a document retriever and a memory recaller. The agent (the LLM) decides which to call.

backend/app/chat/langgraph_agent.py

```

from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.base import BaseCheckpointSaver
from langgraph.graph.state import CompiledStateGraph

from .prompts import SYSTEM_PROMPT
  
```

```

from .tools import tools    # [retrieve_user_documents, recall_memories, (tavily)]

def build_retrival_graph(
    checkpointer: BaseCheckpointSaver, model_name: str
) -> CompiledStateGraph:
    """Compile the memory-augmented ReAct agent.

    The `checkpointer` provides SHORT-TERM memory: it persists the running
    message list per `thread_id`, so the agent resumes mid-conversation.
    The `tools` provide the LONG-TERM read paths: document retrieval and
    cross-session memory recall. Together they form the loop in Figure 1.1.
    """
    model = create_model(model_name=model_name)
    agent = create_react_agent(
        model=model,
        tools=tools,
        prompt=SYSTEM_PROMPT,    # ReAct routing policy (which tool, when)
        checkpointer=checkpointer,
    )
    return agent

```

The memory read path itself is a single tool. Note that it is scoped by `user_id` pulled from the run configuration — this is where per-user isolation begins (Chapter 8).

backend/app/chat/tools.py - the memory read path

```

@tool
async def recall_memories(query: str, config: RunnableConfig) -> str:
    """Search the user's episodic, semantic, procedural, and entity memory
    for background context before answering."""
    user_id = config["configurable"].get("user_id")
    if not user_id:
        return "No personalized memory session is active."

    from app.db.main import async_session
    from app.memory.reader import search_memories

    async with async_session() as session:
        results = await search_memories(
            session=session,
            user_id=str(user_id),
            query=query,
            strategy="combined",    # recency x importance x relevance (Ch. 5)
            limit=5,
        )

    if not results:
        return "No relevant user memories found."

    out = []
    for item in results:

```

```

m = item["memory"]
out.append(f"- [{m.memory_type.upper()}] {m.content} "
          f"(Importance: {m.importance_score})")
return "\n".join(out)

```

Two design choices in this small function are worth naming explicitly, because they recur throughout the book:

1. **Memory recall is a tool, not an always-on prefix.** The agent calls it only when the system prompt’s routing policy decides the query is about the user. This keeps the context window lean (Chapter 7) and avoids paying retrieval latency on every turn.
2. **The query is scored with the combined strategy by default.** Memory recall is never pure similarity — it blends recency and importance so that a fresh, salient fact outranks an old, trivial one even when both match the query (Chapter 5).

✓ Best Practice

Make memory recall an *explicit, auditable* tool call rather than a hidden prompt injection. When recall is a tool, every retrieval is logged, attributable to a query, and visible in the agent’s trace — which is exactly what you need when debugging “why did the agent think I live in Boston?” in production.

1.5 Test It in the UI

Project Code — Lab 1 • route /labs/1

Goal: see the stateless-vs-stateful gap with your own eyes.

1. Start the project (see *How to Use This Book*) and sign in.
2. Open `/labs / 1`. The lab runs your prompt twice: once against the stateless endpoint (`POST /chat/`) and once against a memory-augmented thread (`POST /chat/{thread_id}`).
3. Enter: “Remember my production server runs on port 8080.” Submit.
4. Now enter: “Which port is my server on?”
5. **Expected:** the *Without memory* column cannot answer (or guesses); the *With memory* column replies “8080,” because the fact was extracted on the first turn and recalled on the second.

What you just verified: persistence is architectural. The only difference between the two columns is whether a write path and a recall tool exist — not the model, not the prompt, not the context length.

Chapter Summary

What: Memory is a per-user read–write subsystem layered onto RAG. It contributes a recall read path before generation and an extraction write path after it.

Why: Stateless retrieval cannot carry facts across turns or sessions, cannot isolate users, and cannot forget. Those are architectural needs that a larger context window does not satisfy.

Rule of thumb: The moment you need writes, per-user isolation, recency weighting, or forgetting, you need memory — not more retrieval.

- Retrieval is a read over a shared, static corpus; memory is a read–write loop over dynamic, per-user state.
- Memory participates at three points: pre-retrieval query shaping, a parallel recall read path, and a post-response write path.
- In the reference agent, recall and document retrieval are *tools*; the LLM routes between them. Short-term memory is the checkpoint; long-term memory is the store behind `recall_memories`.
- Memory recall defaults to a combined (recency $\times$ importance $\times$ relevance) score, never pure similarity.

1.6 Evaluation

Ten questions covering the chapter. Work them before reading the answers that follow.

Exercise 1.1 — Concept

In one sentence each, distinguish retrieval from memory along *mutability* and *scope*.

Hint. *One is read-only and shared; the other is read–write and identity-scoped.*

Exercise 1.2 — Concept

Name the three points in the RAG pipeline where memory participates, and give an example operation at each.

Hint. *Think before retrieval, beside retrieval, and after generation.*

Exercise 1.3 — Debugging

A teammate “fixes” the missing-port problem by raising the context window from 8k to 128k tokens. Why does the bug reappear the next day?

Hint. *What happens to the window when the session ends?*

Exercise 1.4 — Production

Why is memory recall implemented as a *tool call* rather than always prepending the user’s memories to every prompt? Give two reasons.

Hint. *Consider context-window cost and per-turn latency.*

Exercise 1.5 — Production

The `recall_memories` tool returns “No personalized memory session is active.” when `user_id` is missing. Which broader architectural property does that guard protect?

Hint. What would happen if recall ran without an identity?

Exercise 1.6 — Debugging

A stakeholder reports the agent “remembers” a fact the user never stated. List two subsystems you would inspect first and why.

Hint. A fact must have been written before it can be recalled.

Exercise 1.7 — Concept

Why does memory ranking blend recency and importance into the score instead of ranking purely by semantic similarity?

Hint. Two memories can match a query equally well yet differ in freshness and salience.

Exercise 1.8 — Production

In the reference agent, what provides short-term memory and what provides long-term memory? Name the concrete components.

Hint. One is a checkpointer keyed by thread; one is a store behind a tool.

Exercise 1.9 — Debugging

You observe that the agent calls `tavily` web search to answer a question about an uploaded document. Where is the routing decision made, and where would you change it?

Hint. The behaviour is governed by a policy, not by code branching in Python.

Exercise 1.10 — Design

Sketch (in words) what you would add to turn a read-only documentation search box into a memory-augmented assistant. List the minimum new components.

Hint. You need a write path, a per-user store, and a recall read path.

Answers

Answer 1.1

Distinguish retrieval from memory along mutability and scope.

Mutability: retrieval is read-only at query time (the corpus is not modified by answering); memory is read *and* written every turn as new facts are extracted. **Scope:** retrieval is shared across all users (the same docs serve everyone); memory is isolated per user/tenant, so one user’s facts never surface for another.

Answer 1.2

Three points where memory participates.

(1) **Pre-retrieval** — query reformulation, where prior context enriches the raw query before any search (see `reformulate_query`, Chapter 5). (2) **Parallel context source** — a recall read path that runs beside document retrieval and contributes per-user facts (the `recall_memories` tool). (3) **Post-response write** — extraction of new memory units from the completed turn (Chapter 4).

Answer 1.3

Why does the bug reappear after raising the context window?

A larger window only enlarges *short-term* working memory *within a single session*. When the session ends (or the window is later trimmed to control cost/latency), everything in it is discarded. The port fact was never persisted to an external store, so the next day’s session starts empty. Persistence requires a write path to durable storage, not a bigger prompt.

Answer 1.4

Why is recall a tool call, not an always-on prefix?

(1) **Context budget**: prepending all memories to every prompt wastes tokens and crowds out documents and the actual question (Chapter 7). (2) **Latency/cost**: memory retrieval (embedding + scoring + a DB round trip) is not free; running it only when the model decides the query is user-specific avoids paying that cost on turns that do not need it. A bonus reason: tool calls are auditable, so every recall is traceable.

Answer 1.5

Which property does the missing-`user_id` guard protect?

Tenant isolation. Memory is identity-scoped; running a recall without a `user_id` would either return nothing useful or, worse, risk querying across users. Refusing to recall without an identity makes per-user isolation a precondition of the read path rather than an afterthought.

Answer 1.6

Agent “remembers” a fact the user never stated.

Inspect (1) the **write path** (`memory/writer.py`): the LLM extractor may have hallucinated or over-generalised a fact from an ambiguous turn, or salience scoring let a low-quality extraction through. (2) the **retrieval relevance threshold** in `reader.py`: a loose threshold can surface a weakly-related memory that the model then states as fact. Both are upstream of generation; the model is only repeating what was written and recalled.

Answer 1.7

Why blend recency and importance instead of pure similarity?

Similarity answers “does this memory match the query?” but not “is it still true and worth surfacing?” Two memories can match equally while one is a fresh, high-salience preference and the other a stale, throwaway remark. Blending recency (decay over time) and importance

(salience score) lets the fresh, salient memory win — the core idea behind the combined score in Chapter 5.

Answer 1.8

Short-term vs. long-term components in the reference agent.

Short-term: the LangGraph *checkpoint* passed to `create_react_agent`, which persists the running message list per `thread_id` so a conversation resumes. **Long-term:** the memory *store* (Postgres + pgvector) read through the `recall_memories` tool and written by the extraction path — it survives across threads, sessions, and restarts.

Answer 1.9

Agent wrongly uses web search for a document question.

The routing decision is made by the **ReAct policy in the system prompt** ([backend/app/chat/prompts.py](#)), not by Python control flow. The prompt already states that web search must *not* be used as a fallback for document questions; if the agent violates this, you tighten that policy text (or constrain the tool set for document-scoped threads), then re-test — you do not add an `if` statement around the tool.

Answer 1.10

Minimum components to make a search box memory-augmented.

At minimum: (1) a **write path** that extracts memory units from each interaction; (2) a **per-user store** (relational + vector) with identity-scoped namespaces; (3) a **recall read path** (a tool) that scores candidates by relevance, recency, and importance; and, to close the loop, (4) short-term **session state** (a checkpoint) so multi-turn tasks stay coherent. Optionally add consolidation and forgetting (Chapters 6 and 10) once the basics work.

Types of Memory

Subscribe → aiengineeringinsider.substack.com/subscribe

"Memory" is not one thing. A production agent juggles a fast, ephemeral scratchpad and several durable stores — and confusing them is the most common design mistake in the field.

Chapter 1 drew the line between retrieval and memory. This chapter draws the lines *within* memory. Borrowing from cognitive science, we separate short-term (working) memory from long-term memory, then split long-term into four operational types — episodic, semantic, procedural, and entity — that map directly onto the `memory_type` column in the reference project's schema. Getting this taxonomy right is what lets you decide *what* to store, *how long* to keep it, and *which* retrieval strategy applies.

2.1 Short-Term vs. Long-Term

Two horizons

Short-term / working memory is the live conversation: the message list in the LLM context window, scoped to one session, managed by a checkpointer. It is fast, ephemeral, and bounded by the context budget.

Long-term memory persists across sessions, users, and restarts. It lives outside the context window in a database and is pulled in on demand.

In the reference project these are two different mechanisms. Short-term memory is the LangGraph checkpointer keyed by `thread_id` (the running messages of a chat thread). Long-term memory is rows in the `memories` table, retrieved by the read path (Chapter 5). A bigger context window grows the former; only a write to the latter creates persistence.

! Pitfall / Warning

Do not use a long context window as a substitute for long-term memory, or a database as a substitute for working memory. The first leaks cost and forgets on session end; the second adds a network round-trip to every reasoning step. Each horizon has its mechanism — keep them separate.

2.2 The Four Long-Term Types

Long-term memory is not monolithic. The schema tags every row with one of four types, because each answers a different question and ages differently.

Type	Answers	Example (from seeded demo data)	Typical lifetime
episodic	"What happened?"	"User mentioned shipping a payments service on June 14."	Decays; useful then fades
semantic	"What is true?"	"User strongly prefers Python over Java for services."	Long-lived; high importance
procedural	"How should I act?"	"Format code answers with type hints."	Long-lived; updated on reflection
entity	"What about X?"	"Production API runs on port 8080."	Until the entity changes

Table 2.1: The four long-term memory types, each with a real example drawn from the project's seeded demo users (`backend/app/db/seed.py`).

2.2.1 Episodic — events

Specific things that happened, time-stamped. They are valuable in the moment (resuming a task, recalling a recent decision) and should decay: an event from six months ago rarely needs to surface. Episodic memories typically carry lower importance and rely on recency in retrieval.

2.2.2 Semantic — facts

Distilled, durable knowledge about the user or domain: preferences, location, role. These are the backbone of personalisation, carry high importance, and should resist decay. "User prefers Python" stays true across sessions.

2.2.3 Procedural — behaviour

How the agent should act: formatting rules, workflow preferences, response style. Unlike the others, procedural memory is often *updated at runtime* via reflection (Chapter 6) — the agent revises its own operating rules.

2.2.4 Entity — structured facts about a thing

Attributes tied to a specific person, place, or object: "the staging cluster is in eu-west-1," "the API port is 8080." Entity memory is where contradictions concentrate (the value changes), which is why it is the main driver of conflict detection (Chapter 6).

★ Key Insight

The type is not cosmetic — it selects the retrieval behaviour. Semantic and procedural memories want *importance*-weighted recall; episodic wants *recency*; entity wants the *latest non-contradicted* value. One column drives four different policies.

2.3 Architecture: the Memory Schema

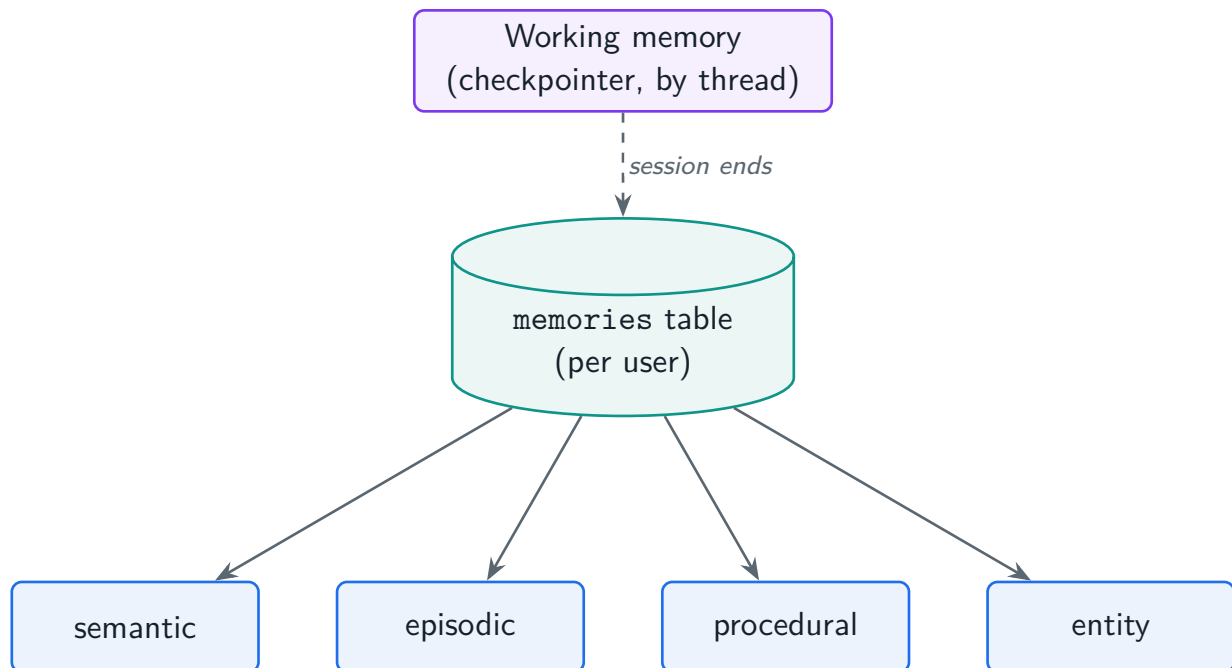


Figure 2.1: The memory taxonomy. One `memories` table holds all four long-term types, distinguished by `memory_type`; short-term working memory is held separately by the checkpointer.

2.4 Production Code: One Table, Four Types

The schema keeps all long-term memory in a single table, typed by a string column, with the metadata each retrieval policy needs: an importance score, a decay rate, an access count, and timestamps.

backend/app/db/models.py - the Memory row

```

class Memory(Base):
    __tablename__ = "memories"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    user_id: Mapped[UUID] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    thread_id: Mapped[UUID] = mapped_column(
        ForeignKey("threads.id", ondelete="CASCADE"), nullable=True)
    memory_type: Mapped[str] = mapped_column(String(50)) #
    ↪ episodic/semantic/procedural/entity
    content: Mapped[str] = mapped_column(String(1000))
    importance_score: Mapped[float] = mapped_column(default=0.5) # salience (Ch. 4)
    access_count: Mapped[int] = mapped_column(default=1)
    last_accessed_at: Mapped[datetime] = mapped_column(
        server_default=func.now(), onupdate=func.now()) # drives recency
    ↪ (Ch. 5)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
    decay_rate: Mapped[float] = mapped_column(default=0.05) # forgetting
    ↪ curve (Ch. 4)
    is_active: Mapped[bool] = mapped_column(default=True) # soft-delete /
    ↪ pruning
  
```

```
is_shared: Mapped[bool] = mapped_column(default=False)           # user vs. global
↪ (Ch. 8)
metadata_json: Mapped[str] = mapped_column(String(2000), default="{}")
```

Three columns are worth highlighting because later chapters depend on them: `importance_score` (salience, set on write), `decay_rate` + `last_accessed_at` (which together yield the recency signal), and `is_active` (soft-delete so a “forgotten” memory can be pruned without losing the audit trail). The `thread_id` is nullable: a memory can be tied to the conversation that produced it or promoted to a user-global fact.

✓ Best Practice

Store all long-term types in one table with a type discriminator rather than four tables. Retrieval scores across types in a single query, conflict detection compares an entity memory against any other memory, and consolidation operates on one stream. Splitting by type prematurely fragments every cross-type operation.

2.5 Test It in the UI

Project Code—Lab 2 • routes `/labs/2` and `/memory`

Goal: create one memory of each type and watch them flow into the store.

1. Log in as a seeded demo user (e.g. `alice@example.com` / `Password123!`). The store is already populated — open `/memory` and note the *Type* badges (episodic, semantic, procedural, entity) on the seeded rows.
2. Open `/labs/2`. Add a memory for each type using the dropdown — e.g. a *semantic* fact (“I prefer dark mode”), an *entity* (“My laptop is a MacBook Pro”), set the importance slider, and save.
3. Return to `/memory`. **Expected:** your new rows appear with the matching colored type badge and the importance you chose.

What you just verified: the type discriminator is a first-class field you set at write time, and the dashboard renders the same four types the schema defines.

Chapter Summary

What: Memory splits into short-term (working, in-context, by thread) and long-term (persistent, in the `memories` table). Long-term has four types: episodic, semantic, procedural, entity.

Why: Each type answers a different question and ages differently, so each maps to a different retrieval policy and lifetime.

Rule of thumb: Events → episodic (decay), durable facts → semantic (importance), behaviour rules → procedural (reflection), thing-attributes → entity (latest value).

- Short-term memory is the checkpointer (per `thread_id`); long-term memory is rows in one typed table.
- The four types share a schema but drive different retrieval and lifetime policies via the importance, decay-rate, and last-accessed columns.
- Entity memory is where contradictions concentrate, feeding conflict detection (Chapter 6).
- `is_active` enables soft-delete/pruning without losing the record.

2.6 Evaluation

Exercise 2.1 — Concept

Differentiate short-term and long-term memory by *mechanism* and *lifetime* in the reference project.

Hint. One is a checkpointer keyed by thread; one is rows in a table.

Exercise 2.2 — Concept

For each of the four long-term types, give the question it answers and one example.

Hint. Happened / is true / how to act / about a thing.

Exercise 2.3 — Design

A user says "Call me Sam, not Samuel." Which memory type is this, and which retrieval signal should dominate its recall?

Hint. It is a durable behaviour/preference, not an event.

Exercise 2.4 — Debugging

Episodic memories from months ago keep surfacing and crowding out current facts. Which two columns would you tune, and how?

Hint. Think decay rate and importance.

Exercise 2.5 — Production

Why does the schema keep all four types in one table instead of four tables? Give two concrete operations that benefit.

Hint. Retrieval scoring and conflict detection.

Exercise 2.6 — Concept

What is the role of `is_active`, and why soft-delete rather than a hard `DELETE`?

Hint. Forgetting vs. auditability.

Exercise 2.7 — Design

`thread_id` is nullable on a memory. Describe one memory that should keep its `thread_id` and one that should be promoted to user-global (`thread_id = NULL`).

Hint. *Conversation-specific context vs. a stable preference.*

Exercise 2.8 — Debugging

A teammate stores a user's allergy as an **episodic** memory. Why is this the wrong type, and what goes wrong at retrieval time?

Hint. *Allergies are durable facts; episodic memories decay.*

Exercise 2.9 — Production

Which type is most prone to contradictions over time, and what schema features support resolving them?

Hint. *Values that change; see the conflict table and `is_active`.*

Exercise 2.10 — Design

Procedural memory is updated at runtime. Sketch the safe write flow for the agent rewriting its own formatting rule.

Hint. *Validate, version, and don't overwrite blindly.*

Answers

Answer 2.1

Short-term vs. long-term: mechanism and lifetime.

Short-term is the LangGraph checkpoint, keyed by `thread_id`; it holds the running message list of one conversation and lives only as long as that session/window. **Long-term** is rows in the `memories` table, scoped by `user_id`; it persists across sessions, threads, and restarts and is pulled in on demand by the read path.

Answer 2.2

Four types: question + example.

Episodic — "what happened?" ("shipped a payments service on June 14"). **Semantic** — "what is true?" ("prefers Python"). **Procedural** — "how should I act?" ("format with type hints"). **Entity** — "what about X?" ("API port is 8080").

Answer 2.3

"Call me Sam."

It is a **procedural** (or semantic) preference about behaviour, not a one-off event. **Importance-weighted** recall should dominate so it surfaces reliably on every relevant turn regardless of how long ago it was stated; recency should not be the deciding factor for a durable preference.

Answer 2.4

Old episodic memories crowd out current facts.

Raise their `decay_rate` so recency falls off faster (episodic memories should fade), and/or ensure their `importance_score` is low so the combined score deprioritises them. The combination makes stale events lose to fresh, salient facts in ranking (Chapter 5).

Answer 2.5

One table vs. four.

A single typed table lets **retrieval** score across all types in one query (no union of four tables), and lets **conflict detection** compare a new entity value against any existing memory uniformly. Consolidation also operates on one stream. Four tables would fragment each of these into multi-table joins.

Answer 2.6

Role of `is_active`.

It is a soft-delete flag: a "forgotten," pruned, or conflict-superseded memory is set `is_active = False` so it stops appearing in retrieval (which filters on `is_active == True`) while the row remains for auditing, undo, and governance. A hard `DELETE` would destroy the audit trail and make right-to-be-forgotten reporting harder to reason about.

Answer 2.7

Nullable `thread_id`.

Keep `thread_id` on a memory like "in this thread the user is debugging the deploy script" — it is context for that conversation. Promote to `thread_id = NULL` a stable fact like "user prefers Python," which is true across every thread and should be recallable everywhere.

Answer 2.8

Allergy stored as episodic.

An allergy is a durable **semantic** fact, not an event. Stored as episodic with a normal decay rate, its recency score falls over time and it drops below the retrieval threshold — so weeks later the agent may stop recalling a safety-critical fact. Type it **semantic** with high importance and low decay.

Answer 2.9

Most contradiction-prone type.

Entity memory — attribute values (ports, regions, names) change over time, producing old-vs-new contradictions. The `memory_conflicts` table links the old and new memory IDs, and `is_active` lets the resolver deactivate the losing version while keeping both on record (Chapter 6).

Answer 2.10

Safe procedural self-update.

(1) The agent proposes a new rule as structured output; (2) **validate** it (parse JSON, sanity-check length/format), falling back to the existing rule on malformed output; (3) **version** it (write a new procedural memory or bump a `schema_version` in metadata) rather than silently overwriting; (4) optionally record the prior rule as inactive so the change is auditable and reversible.

Memory Storage Architectures

Subscribe → aiengineeringinsider.substack.com/subscribe

Where you put a memory determines how you can ask for it back. The store is not an implementation detail — it is a design decision with consequences for recall, consistency, and cost.

A memory system needs to answer three different kinds of question: "what is *similar* to this?" (semantic recall), "what is the *exact* value of this field?" (structured lookup), and "what is *connected* to this?" (relationships). No single store does all three well. This chapter surveys the storage options — vector, key-value/relational, and graph — explains the hybrid and tiered patterns that combine them, and grounds it all in the reference project, which pairs PostgreSQL (relational) with pgvector (vector) in one database.

3.1 Three Questions, Three Store Types

Store	Best at	Examples & notes
Vector DB	Semantic similarity ("find related")	Pinecone, Weaviate, Qdrant, Milvus, pgvector. ANN index over embeddings.
Key-value Doc	Exact lookup, profiles	Redis, DynamoDB, MongoDB, a SQL row. O(1) by key; no fuzzy match.
Relational	Structured facts, filters, joins	PostgreSQL. Strong consistency, rich queries, transactions.
Graph DB	Relationships, multi-hop	Neo4j, Memgraph. "Who introduced whom"; traversal-heavy recall.

Table 3.1: Each store class is shaped to a different question. Production memory systems usually combine at least two.

Why not just a vector DB?

Embedding everything and searching by similarity is seductive but lossy. "What is the user's API port?" has an *exact* answer that similarity search can only approximate; "is this user a teammate of that user?" is a *relationship* a vector index cannot represent. Similarity is one access pattern among several.

3.2 Hybrid and Tiered Storage

3.2.1 Hybrid: each fact in the store that fits it

A hybrid architecture routes each memory to the store matching its access pattern: durable rows in relational, embeddings for similarity in a vector index, relationships in a graph. The reference project is a *relational + vector* hybrid: the `memories` table is the source of truth (Chapter 2), and a parallel pgvector index holds an embedding of each memory’s content for semantic recall. Crucially, both live in one PostgreSQL instance, so there is no second datastore to operate.

3.2.2 Tiered: hot → warm → cold

Orthogonal to store *type* is store *temperature*. Frequently-needed memory should be cheap to reach; rarely-needed memory can be slow and cheap to hold:

Hot In the context window right now — zero retrieval latency, highest token cost. The working set.

Warm A cache or fast store (Redis, an in-process index) — single-digit millisecond recall for the user’s active memories.

Cold The durable database (Postgres + pgvector) — authoritative, cheapest per byte, higher latency.

★ Key Insight

Store *type* answers “how do I query it?”; store *temperature* answers “how fast and how often?”. A mature design picks both axes per memory — e.g. a hot, cached *semantic* profile plus cold, on-demand *episodic* history.

3.3 Graph-Based Memory: Knowledge Graph Triples

While similarity search handles fuzzy matches and key-value lookups handle exact parameters, relationship-rich facts require representing memory as a network. Storing experiences as a flat list of text strings or independent embeddings loses the structural topology of how facts connect. This is resolved by representing memory as a set of **Knowledge Graph Triples**:

$$(S, P, O) \rightarrow (\text{Subject}, \text{Predicate}, \text{Object})$$

where nodes represent entities (people, concepts, locations) and edges represent typed directed relationships.

3.3.1 Why Graph representation matters

Consider the facts: “Alice works with Bob” and “Bob is the lead of the DevOps team.” To answer: “Which team is Alice connected to?”, a similarity search over independent memories requires two separate queries and a manual stitching step (multi-hop reasoning). A Graph database (such as Neo4j or Memgraph) traverses these edges in a single traversal operation ($O(1)$ relationship-hop latency), scaling independently of the total text document count (Robinson et al., 2015).

3.3.2 Triple Extraction & Integration

Below is a sample python integration showing how an agent extracts structured entity triples from raw conversation turns using schema-constrained outputs, preparing them to be indexed into a graph memory database.

backend/app/memory/graph_extractor.py - SPO Triple Extraction

```
from pydantic import BaseModel, Field
from typing import List
from langchain_core.messages import BaseMessage

class MemoryTriple(BaseModel):
    subject: str = Field(description="The primary subject entity (e.g., 'Alice')")
    predicate: str = Field(description="The relationship verb or attribute link
    ↪ (e.g., 'works_at')")
    object: str = Field(description="The target object entity or value (e.g., 'Google
    ↪ DeepMind')")

class GraphMemoryExtraction(BaseModel):
    triples: List[MemoryTriple] = Field(description="List of extracted fact triples.")

async def extract_graph_triples(conversation: List[BaseMessage], model) ->
    ↪ List[MemoryTriple]:
    """
    Extracts Subject-Predicate-Object (SPO) triples from dialogue turns.
    These triples are indexed into a graph database to enable multi-hop reasoning.
    """
    dialogue_str = "\n".join([f"{m.type}: {m.content}" for m in conversation])
    prompt = (
        f"Analyze the following conversation and extract all stable relationship
        ↪ triples:\n"
        f"{dialogue_str}\n\n"
        f"Focus on entity connections, facts, and persistent relations. Output
        ↪ structured JSON."
    )
    # Configure the LLM to output schema-conforming JSON
    structured_model = model.with_structured_output(GraphMemoryExtraction)
    res = await structured_model.ainvoke(prompt)
    return res.triples
```

3.3.3 Neuro-Symbolic Memory: Vectors + Logic

While vector databases represent memory as continuous vectors and graph databases represent them as discrete symbols, **Neuro-Symbolic Memory** combines both paradigms. It overlays neural representations (dense vector similarity) with symbolic rules and logical constraints (first-order logic reasoners). For instance, when a neural search retrieves: *"Alice works in the Google deep learning branch,"* a symbolic reasoner runs logical constraints to verify if this conflicts with a hard rule like: *"All deep learning branch staff are located in London,"* resolving reasoning drifts or hallucinations before context injection (Garcez et al., 2019).

★ Key Insight

Neuro-Symbolic memory acts as an execution-time validation layer. Where embeddings match text by fuzzy semantic intent (which is prone to hallucinating connections), the symbolic constraint engine enforces hard deterministic constraints. The reference project implements this dual approach by filtering semantic recalls with user-defined logical rules before injection.

backend/app/memory/neuro_symbolic.py - Symbolic Rules Verification

```
from typing import List, Dict, Any

class SymbolicConstraintEngine:
    """
    Verifies that neural memory retrieval candidates do not violate hard symbolic
    ↪ rules.
    """
    def __init__(self, rules: List[Dict[str, Any]]):
        self.rules = rules

    def verify_and_filter(self, retrieved_memories: List[Dict[str, Any]]) ->
    ↪ List[Dict[str, Any]]:
        valid_memories = []
        for mem in retrieved_memories:
            content = mem["memory"].content
            is_valid = True

            # Simple symbolic constraint checks
            for rule in self.rules:
                if rule["trigger"] in content.lower():
                    # If memory violates the negative constraint, drop it
                    if rule["forbidden_sub"] in content.lower():
                        is_valid = False
                        break

            if is_valid:
                valid_memories.append(mem)
        return valid_memories
```